Formal Methods in Computer-Aided Design 2026

Interpolation-based Transition Invariant Generation for Proving and Disproving Termination

Konstantin Britikov Martin Blicha Grigory Fedyukovich Natasha Sharygina

Abstract—**Termination and nontermination of infinite-state systems are complementary problems that, despite their close connection, are typically addressed by separate techniques with incompatible witnesses. The core idea of this paper is to utilise the intermediate products of termination and nontermination analysis to mutually guide each other. We present a new termination analysis approach based on the interpolation-based construction of well-founded transition invariants. The new technique leverages Craig interpolation for transition invariant generation, capturing the structural reason for termination. Proposed technique extends safety-based nontermination analysis, allowing it to prove both termination and nontermination within a unified framework. We implemented our approach in Golem and evaluated it on benchmarks from the Termination Competition (TermComp). Empirical results demonstrate the benefit of combininig termination and nontermination analysis, and competitive performance of our technique compared to state-of-the-art tools.**

I. Introduction

Termination analysis of infinite-state systems is a fundamental problem in formal methods. Proving termination and nontermination over LIA is generally undecidable [8], yet many practical techniques succeed on large classes of real-world programs.

Modern techniques treat termination and nontermination as separate problems, since each has a different proof procedure. State-of-the-art termination techniques centre around the synthesis of ranking functions for the transition relation [26], [20], [2], [17], [16], [28], [1], [31]. The existence of a ranking function witnesses termination by providing a measure that strictly decreases along every execution. Alternative approaches are based on the fixpoint computation [30], and the synthesis of disjunctively well-founded transition invariants [3], [13], [19], [29]. Nontermination analysis techniques are mostly centered around the detection of recurrent sets — sets of states from which execution cannot escape [1], [11], [15], [9], [10], [21], [4]. There also exist alternative nontermination approaches based on Hoare-style reasoning [22], safety proving [11], [4], or representing infinite runs as sums of geometric series [23].

In this work, we present a unified framework that is capable of proving both termination and nontermination. This technique is constructed by extending the safety-based nontermination analysis [11] (SNA), allowing it to additionally prove termination. Implemented termination analysis is based on a new interpolation-based method for generation of disjunctively well-founded transition invariants, which utilises the intermediate results of nontermination analysis. Proposed approach uses terminating traces constructed by SNA, overapproximating them with Craig's interpolation. These overapproximations are used as transition invariant candidate, which can prove termination if it is a valid transition invariant over all reachable states. Additionally, algorithm computes states over which candidate transition invariant is not valid. Those states are separately analyzed for termination and nontermination, introducing smaller subproblem, compared to the analysis of the whole transition system.

Our new technique was implemented in Golem CHC Solver. We evaluated our technique over the Integer Transition Systems benchmarks from the Termination Competition. Proposed approach that combines termination and nontermination frameworks demonstrated significant improvement for nontermination analysis, compared to the original SNA algorithm, while also solving a large number of terminating instances. The comparison with winners of Termination Competition 2025 (KoAT, LoAT [24], T2 [9]) demonstrated that the developed approach is competitive with the state-of-the-art tools, and is able to solve unique instances, including instances that were never solved in the history of Termination competition.

Overall, this paper presents following contributions: (1) a new interpolation-based method for the synthesis of disjunctively well-founded transition invariants, (2) a unified termination/nontermination procedure, which utilises the intermediate results of analysis for self-guidance and (3) an implementation of the described procedure in Golem and evaluation of proposed approach.

II. Preliminaries

This section introduces the definitions and formalisms used in the paper. Logical formulas in our approach are defined over the theory of Linear Integer Arithmetic (LIA). We use X, X' to denote set of variables (unprimed and primed), where $X' = \{x' | x \in X\}$, X is a set of state variables, and X' is a set of next state variables. If there are variables of many different states in the formula we use superscript notation, e.g., $X^{(0)}, \ldots, X^{(n)}$.

The formula defined over X is a state formula, and a formula over $X \cup X'$ is a transition formula. State formulas encode a set of states, and transition formulas define binary relations over the state. Using $Id(X, X')$ we denote a transition formula $\bigwedge_{x \in X} x = x'$.

Transition systems and Safety Problem: A *transition system* (TS) is defined as $TS = \langle Init, Tr, X \rangle$, where $Init$ is a formula encoding the initial states of the system, Tr is a transition formula, which encodes the transition relation, and X is a set of system's variables. With respect to transition system, a *transition trace* is a formula encoding multiple consecutive

https://doi.org/

transitions: $Tr^n(X^{(0)}, \dots, X^{(n)}) = Tr(X^{(0)}, X^{(1)}) \wedge \dots \wedge Tr(X^{(n-1)}, X^{(n)})$. We write $Tr^n(X^{(0)}, \dots, X^{(n)})$ for this conjunction throughout the paper. A *safety problem* is defined as $P = \langle Init, Tr, Bad, X \rangle$ where Bad is a formula that represents states that violate the safety property.

Safety verification amounts to determining whether there exists a transition trace that reaches a Bad state from the initial states. If TS is safe, then there exists an inductive invariant $Inv(X)$ s.t. (1) $\forall X. Init(X) \rightarrow Inv(X)$, (2) $\forall X, X'. Inv(X) \wedge Tr(X, X') \rightarrow Inv(X')$, and (3) $\forall X. Inv(X) \rightarrow \neg Bad(X)$. We use $safetyCheck(P)$ to denote a safety verification procedure that returns a tuple $\langle SAFE, Inv(X) \rangle$ if TS is safe ($Inv(X)$ is inductive invariant). Otherwise, if TS is unsafe it returns $\langle UNSAFE, Tr^n(X^{(0)}, \dots, X^{(n)}) \rangle$, such that $\exists X^{(0)}, \dots, X^{(n)}$.

$$Init(X^{(0)}) \wedge Tr^n(X^{(0)}, \dots, X^{(n)}) \wedge Bad(X^{(n)})$$

Nontermination: A transition system is called **nonterminating** if there exists an initial state, from which it is possible to take any number of transitions:

$$\exists X. Init(X) \wedge \forall n \in \mathbb{N}. \exists X^{(0)}, \dots, X^{(n)}. Tr^n(X, X^{(1)}, \dots, X^{(n)})$$

Definition 1 (Recurrent Set)**.** For a Transition System $\langle Init, Tr, X \rangle$, a formula $N(X)$ is a recurrent set if: (1) $\forall X. N(X) \rightarrow \exists X'. Tr(X, X') \wedge N(X')$, (2) $\exists X. N(X) \wedge Init(X)$.

Transition System is nonterminating if there exists a *recurrent set*. Intuitively, every state in the recurrent set has a successor that also lies in the recurrent set, and the recurrent set contains some of the initial states.

Termination: A TS is called **terminating** if for every initial state it is possible to take only limited number of transitions. For a Transition System $\langle Init, Tr, X \rangle$, $\forall X$.

$$Init(X) \rightarrow \exists n \in \mathbb{N}. \neg \exists X^{(0)}, \dots, X^{(n)}. Tr^n(X, X^{(1)}, \dots, X^{(n)})$$

Definition 2 (Transition invariant)**.** For a Transition System $\langle Init, Tr, X \rangle$, $TrInv(X, X')$ is a transition invariant if: $\forall X, X'. R(X) \wedge Tr^+(X, X') \rightarrow TrInv(X, X')$, where $Tr^+(X, X')$ is the transitive closure of $Tr(X, X')$ and $R(X)$ is a formula encoding all the states, reachable within transition system.

Definition 3 (Well-founded Relation)**.** A binary relation $R \subseteq D \times D$ over a domain D is *well-founded* if there exists no infinite descending chain:

$$\nexists\, d_0, d_1, d_2, \dots \in D.\ \forall i \in \mathbb{N}.\ (d_i, d_{i+1}) \in R$$

A relation $R(X, X')$ can be proven to be well-founded [26] by synthesizing a ranking function $f(X)$, such that $\forall X, X'. R(X, X') \rightarrow f(X) > f(X')$.

Transition invariant [27] can be used to prove termination. If transition invariant is a finite union of well-founded relations (for example if every separate relation has a ranking function), then the Transition System terminates.

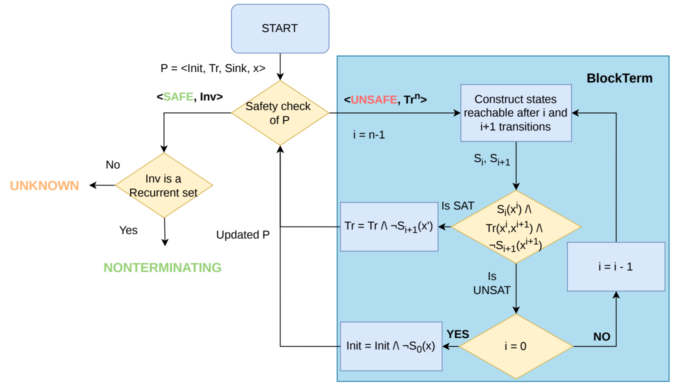

Figure 1. Execution flow of Safety-based Nontermination analysis, cf [11]

Definition 4 (Disjunctively well-founded relation)**.** A relation $D(X, X') = \cup_{i=0}^{n} R_i(X, X')$ is called disjunctively well-founded if every relation $R_i(X, X')$ is well-founded.

Craig Interpolation: An abstraction technique [14], such that given two formulas A and B such that $A \wedge B$ is unsatisfiable, a Craig interpolant for A and B is a formula I such that $A \rightarrow I$, $I \wedge B$ is unsatisfiable, and I only contains the symbols, common to both A and B. We use $Itp(A, B)$ to denote the computation of the Craig interpolant for A and B.

A. *Proving Nontermination via Safety*

Many techniques reduce termination and nontermination analysis to safety verification. One notable approach is the reduction of nontermination to safety, which constructs strong recurrent sets via inductive invariants [11]. It was introduced in the context of proving nontermination of programs with loops, and can be extended for the Transition Systems. This approach is described below.

Definition 5 (Sink states)**.** For a Transition System $\langle Init, Tr, X \rangle$, a state s is a sink state, if $\neg \exists s'. Tr(s, s')$. We use $Sink(X)$ to denote the formula encoding all sink states in the transition system, such that $\forall X. Sink(X) \leftrightarrow \neg \exists X'. Tr(X, X')$

Algorithm 1 for Safety-based Nontermination analysis (SNA) takes a safety problem $P = \langle Init, Tr, Sink, X \rangle$, where $Sink(X) = QE(\exists X'. \neg Tr(X, X'))$. As an output, it returns NONTERM if the system is nonterminating, and UNKNOWN, if the algorithm cannot conclude the analysis. Figure 1 contains the execution flow of Safety-based Nontermination.

The approach executes a safety check of P in a loop. At line 2, the algorithm handles the case when P is unsafe (i.e., exists a trace reaching sink states). In this case, the algorithm calls the BLOCKTERM procedure (Algorithm 2), which refines the safety problem P by blocking the states that deterministically lead to the sink states, updating the Safety problem. BLOCKTERM is described in detail below. In the case when P is safe, algorithm checks if inductive invariant $Inv(X)$ is a recurrent set (line 6). The check at

Algorithm 1: Safety-based Nontermination Analysis (SNA), cf [11]

```
  Input  : Safety problem P = ⟨Init, Tr, Sink, X⟩
  Output: NONTERM | UNKNOWN
1 while ⊤ do
2   ⟨res, φ = Tr^n or Inv⟩ ← safetyCheck(P)
3   if res = UNSAFE then
4     P ← BlockTerm(P, φ)
5   else
6     if ∀X.φ(X) → ∃X'.Tr(X, X') then
7       return NONTERM
8     return UNKNOWN
```

line 6 is executed using quantifier elimination, checking if the following formula is unsatisfiable:

$$\exists X.\phi(X) \wedge \neg QE(\exists X'.Tr(X, X')).$$

If $Inv(X)$ is a recurrent set, then the transition system is nonterminating. Otherwise, algorithm returns UNKNOWN, as the analysis is inconclusive.

Algorithm 2: BlockTerm (P, Tr^n), cf [11]

```
   Input  : Safety problem P = ⟨Init, Tr, Sink, X⟩,
            terminating trace Tr^n(X^(0), ..., X^(n)) =
            Tr(X^(0), X^(1)) ∧ ··· ∧ Tr(X^(n-1), X^(n))
   Output: Refined safety problem P'
   Data   : S_0, ..., S_n - formulas which encode states within
            trace reachable in 0, ..., n transitions
 1 i ← n − 1
 2 while i > 0 do
 3   S_{i+1}(X^(i+1)) ←
       QE(∃X^(0), ..., X^(i), X^(i+2), ..., X^(n).
       Init(X^(0)) ∧ Tr^n ∧ Sink(X^(n)))
 4   S_i(X^(i)) ← QE(∃X^(0), ..., X^(i-1), X^(i+1), ..., X^(n).
       Init(X^(0)) ∧ Tr^n ∧ Sink(X^(n)))
 5   if SAT ? S_i(X) ∧ Tr(X, X') ∧ ¬S_{i+1}(X') then
 6     Tr'(X, X') ← Tr(X, X') ∧ ¬S_{i+1}(X')
 7     return ⟨Init, Tr', Sink, X⟩
 8   i ← i − 1
 9 Init'(X^(0)) ← Init(X^(0)) ∧
     ¬QE(∃X^(1), ..., X^(n).Init(X^(0)) ∧ Tr^n ∧ Sink(X^(n)))
10 return ⟨Init', Tr, Sink, X⟩
```

The BlockTerm procedure takes as input a safety problem P and a trace formula $Tr^n(X^{(0)}, \ldots, X^{(n)})$ such that $Init(X) \wedge Tr^n(X, \ldots, X') \wedge Sink(X')$ is satisfiable. The algorithm iteratively analyses the transition trace from the sink. It checks if it is possible to take a transition that does not lead to the sink states. Line 5 checks if $S_i(X) \wedge Tr(X, X') \wedge \neg S_{i+1}(X')$ is satisfiable, sending a query to the SMT solver. If it is possible, then the algorithm blocks the states that deterministically lead to the sink states at line 6. Otherwise, if such states do not exist, the algorithm blocks the subset of initial states that deterministically lead to the sink states (line 9).

Lemma 1. *If Algorithm 1 returns NONTERM, then the transition system is nonterminating, cf [26].*

Example 1. Consider a transition system $TS = \langle Init, Tr, X\rangle$ with $X = \{x\}$, where x is an integer variable, $Init(X) \triangleq \top$, and $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge x \neq 0$. The sink states are $Sink(x) \triangleq v = 0$, and the initial safety problem is $P = \langle Init, Tr, Sink, x\rangle$. This transition system is nonterminating, since x can decrease indefinitely through negative values.

The execution of Algorithm 1 updates initial states and transition in two iterations of the algorithm, using BlockTerm $Init(X) \leftarrow x \neq 0$, and $Tr(X, X') \leftarrow Tr(X, X') \wedge x' \neq 0$. The safety check of the updated problem concludes SAFE, with $Inv(X) \triangleq x \neq 0$. $Inv(X)$ is a recurrent set, and algorihtm returns NONTERM.

Motivation for transition invariant-based termination analysis. Consider a small modification (replacing $x \neq 0$ with $x > 0$): $Tr(X, X') \triangleq (x' = x - 1 \vee x' = x - 2) \wedge x > 0$. This renders TS terminating, since x is strictly positive and decreases every step. However, Algorithm 1 cannot detect termination by design, in the case of this particular example it would return $UNKNOWN$.

Algorithm 1, would proceed with the execution in a similar manner. First, using BlockTerm it would block initial states that lead to instant termination: $Init(X) \leftarrow x > 0$. In the second iteration, $safetyCheck(P)$ returns UNSAFE with a trace $Tr^1(X^{(0)}, X^{(1)})$, detecting that $\exists X^{(0)}, X^{(1)}.Init(X^{(0)}) \wedge Tr^1(X^{(0)}, X^{(1)}) \wedge \neg Sink(X^{(1)})$. Our key insight is to generalise such traces into a formula describing an entire class of terminating behaviours. If this generalisation yields a disjunctively well-founded transition invariant over all reachable states, it witnesses termination. Particularly in this example, the formula $TrInv(X, X') \triangleq x' < x \wedge x > 0$ is well-founded transition invariant which is sufficient to prove termination of TS. Section III presents the algorithm that automates this generalisation via Craig interpolation.

III. Termination Analysis via Interpolation-based Transition Invariant Generation

We propose a new approach for proving termination that leverages safety verification to generate a well-founded transition invariant candidate via interpolation. At each iteration, our procedure constructs a trace from the initial states to a sink state. A well-founded transition invariant is then constructed based on the overapproximation of the terminating trace. If the invariant is valid for all reachable states, termination is concluded. The interpolation-based approach enables efficient property-directed transition invariant generation, as it is constructed with respect to the termination condition.

A. Interpolation-based Generation of Well-Founded Transition Invariants

The core of our approach is the procedure for automated, interpolation-based generation of well-founded transition invariants: TrInvGen. Given a transition system and a terminating trace, the procedure constructs a disjunctively well-founded approximation of the transition trace. If the resulting

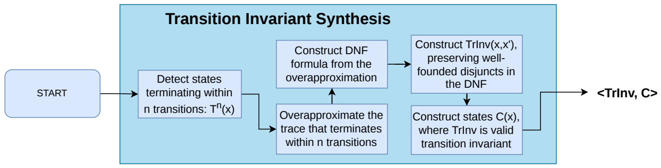

Figure 2. Execution Flow of Interpolation-based Transition Invariant Generation

Algorithm 3: TRINVGEN($P, Tr^n, TInv$).

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$, transition trace $Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)})$, candidate transition invariant $TInv(X, X')$

Output: Updated transition invariant $TInv(X, X')$ and formula $C(X)$ encoding states covered by transition invariant

1 $NT^n(X^{(0)}) \leftarrow QE(\exists X^{(1)}, \ldots, X^{(n)}.$
$\quad Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)}))$

2 $itp(X, X') \leftarrow$
$\quad Itp(Tr^n(X, \ldots, X'), \neg NT^n(X) \wedge \neg Sink(X'))$

3 $Cs(X, X') \leftarrow toDNF(itp)$

4 $TInv \leftarrow TInv \vee \bigvee_i \{d_i \in Cs | \exists$ ranking function $f(X)\}$

5 $C(X) \leftarrow \neg QE(\exists X', X''. \left(Id(X, X') \vee TInv(X, X')\right) \wedge$
$\quad Tr(X', X'') \wedge \neg TInv(X, X''))$

6 **return** $\langle TInv, C\rangle$

formula constitutes a transition invariant over all reachable states, it witnesses termination.

The Procedure consists of five major steps: (1) computing NT^n, the formula encoding states that do not necessarily terminate within n transitions. NT^n is used to construct the interpolant. The negation of NT^n defines the states that necessarily reach $Sink$ in at most n transitions; (2) constructing an over-approximation of the terminating trace via interpolation. The usage of sink states allows the interpolant to capture the reason for termination; (3) converting the interpolant into an underapproximating DNF formula. This allows to efficiently synthesise ranking functions for separate disjuncts; (4) filtering disjuncts to retain only those encoding well-founded relations, constructing a candidate transition invariant. Only well-founded relations are saved, as transition invariant needs to be disjunctively well-founded to witness termination; and (5) computing the formula encoding states for which candidate formula is a valid transition invariant. Transition invariant needs to be valid only over the reachable states to conclude termination. Therefore, if all reachable states satisfy this formula - TS is terminating. The overall flow of the procedure is illustrated in the Figure 2.

Algorithm 3 takes as an input a safety problem $P = \langle Init, Tr, Sink, X\rangle$, a transition trace $Tr^n(X^{(0)}, X^{(1)}, \ldots, X^{(n)})$, and a transition invariant formula $TInv(X, X')$. It returns a tuple of formulas $\langle TInv, C\rangle$, where $TInv(X, X')$ is a disjunctively well-founded transition invariant candidate, and $C(X)$ is a formula encoding the states for which $TInv$ is a valid transition invariant. It proceeds in the following steps:

Step 1: *Computing NT^n (line 1).* $NT^n(X)$ encodes all states that do not necessarily terminate in n transitions, is defined by the following property:

$$NT^n(X^{(0)}) \leftrightarrow \exists X^{(1)}, \ldots, X^{(n)}. Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

NT^n is constructed with quantifier elimination, eliminating all variables from the formula except for $X^{(0)}$. Construction of $NT^{(n)}$ enables the overapproximation of the trace.

Example 2. Consider $TS = \langle Init, Tr, X\rangle$ with $Init \triangleq x > 0$, $Tr(X, X') \triangleq ((x' = x - 1 \vee x' = x - 2) \wedge x > 0) \vee (x < -10 \wedge x' = x - 1)$, $X \triangleq \{x\}$. In this TS, sink states are $Sink(X) \triangleq x \leq 0 \wedge x \geq -10$. For $n = 2$ quantifier elimination would yield $NT^2(X) = x > 2$. States with $x \in \{3, 4\}$ can reach $Sink$ in two transitions, but not necessarily. Therefore these states are included in NT^2.

Step 2: *Interpolation (line 2).* The following formula is unsatisfiable by construction, since any state in $\neg NT^n$ must reach $Sink$ within n steps:

$$\neg NT^n(X^{(0)}) \wedge Tr^n(X^{(0)}, \ldots, X^{(n)}) \wedge \neg Sink(X^{(n)})$$

This allows to compute the **Craig interpolant**, which over-approximates the trace Tr^n. Crucially, the usage of the sink states for interpolant construction allows the interpolant to capture the reason *why* the particular trace is terminating.

Step 3: *DNF underapproximation (line 3).* For a transition invariant to witness termination, it must be disjunctively well-founded, meaning every disjunct must encode a well-founded relation. Converting the interpolant to a full DNF would make each disjunct a conjunction, simplifying the well-foundedness check, but full DNF conversion can cause an exponential blowup in formula size. Moreover, even a complete DNF is not guaranteed to yield a transition invariant or to witness termination.

Instead, our algorithm follows a lazy approach: it constructs an under-approximating DNF formula on demand. Concretely, algorithm constructs disjuncts as follows: (1) SMT solver prodices a model that satisfies itp, (2) Using produced model, the satisfied literals are extracted from the SMT formula and conjoined, (3) This conjunction is one of the dinsjuncts in the constructed DNF. It is blocked from the itp, and the dnfization proceeds (until reaching certain limit). The result is a formula:

$$Cs(X, X') \triangleq \bigvee_{i=0}^{m} d_i$$

where each d_i is a conjunction, and $Cs(X, X') \rightarrow itp(x, x')$. This approach significantly reduces formula size and speeds up both DNFization and subsequent well-foundedness check, while preserving correctness: in the worst case, Cs fails to be a transition invariant over all reachable states, but this is equally possible even with a complete DNF conversion.

Step 4: *Filtering Well-Founded Disjuncts (line 4).* For each disjunct $d(X, X') \in Cands(X, X')$ algorithm checks if $d(X, X')$ encodes a well-founded relation. Our technique uses automated linear ranking function synthesis [26] for checking the existence of a ranking function. The disjuncts for which a ranking function exists are preserved, and the rest are discarded. The resulting formula is disjoined with the input candidate transition invariant $TInv$.

Step 5: *Synthesis of the states for which transition invariant is valid (line 5).* If $TrInv$ is a transition invariant over all reachable states, then TS terminates, since every disjunct corresponds to well-founded relation. To verify this, procedure computes the formula $C(X)$ encoding the states over which $TrInv$ is a valid transition invariant.

Definition 6 (Coverage formula)**.** Consider a transition system $TS = \langle Init, Tr, X\rangle$ and a formula $TrInv(X, X')$. We say that $C(X)$ is a coverage formula, if the following properties hold $\forall X, X', X''$:

$$\begin{aligned} C(X) \wedge Tr(X, X') &\rightarrow TrInv(X, X') \\ C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') &\rightarrow TrInv(X, X'') \end{aligned}$$

Meaning, that $TrInv$ is a transition invariant over states satisfying $C(X)$.

Similarly to NT^n, $C(X)$ is constructed by applying quantifier elimination, negating the states for which $TrInv$ is not a transition invariant.

Example 3. Continuing Example 2, suppose the procedure constructs $TrInv \triangleq x' < x \wedge x' \geq 0$. Quantifier elimination at line 4 then yields $C(X) \triangleq x \geq 0$.

Note. If $\neg C(X)$ is not reachable, then $TrInv(X, X')$ is a disjunctively well-founded transition invariant over all reachable states, and witnesses the termination of transition system. For illustration, transition invariant in the Example 3 is sufficient to prove termination, since $\neg C(X) \triangleq x < 0$ cannot be reached within TS.

Lemma 2. *Let $TS = \langle Init, Tr, X\rangle$ be a transition system and let $C(X)$ be a formula such that $\neg C(X)$ is unreachable in TS. If $TrInv(X, X')$ is a finite disjunction of formulas encoding well-founded relations and satisfies $\forall X, X', X''$:*

$$\begin{aligned} C(X) \wedge Tr(X, X') &\rightarrow TrInv(X, X') \\ C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') &\rightarrow TrInv(X, X'') \end{aligned}$$

then TS is terminating.

Proof. We show that $TrInv$ is a disjunctively well-founded transition invariant over the reachable states of TS, from which termination follows by [27].

Disjunctive well-foundedness. By construction, for each disjunct d_i of $TrInv = \bigvee_i d_i$ there exists a ranking function. Therefore, every disjunct is a logical encoding of well-founded relation [26]. Hence $TrInv$ is disjunctively well-founded.

Transition invariant. We show that $\forall X, X'.\ R(X) \wedge Tr^+(X, X') \rightarrow TrInv(X, X')$, where $R(X)$ denotes the reachable states of T, by induction on the number of steps n in $Tr^+(X, X')$.

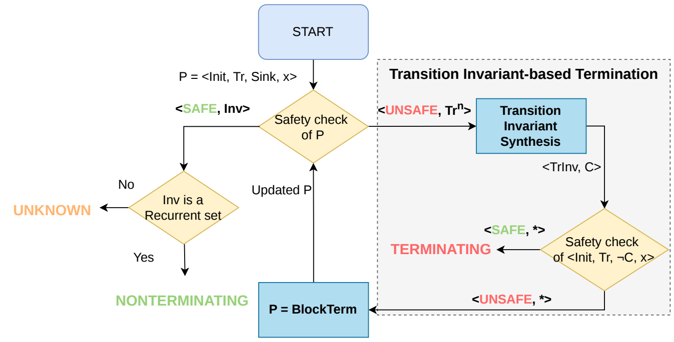

Figure 3. Execution Flow of Interpolation-based Transition Invariant Generation (New components are placed in a grey block, lines 5,6,10).

Base case ($n = 1$): It is needed to prove that:

$$R(X) \wedge Tr(X, X') \rightarrow TrInv(X, X')$$

Since $\neg C(X)$ is unreachable, $R(X) \rightarrow C(X)$.

The following assumption holds by the lemma statement:

$$C(X) \wedge Tr(X, X') \rightarrow TrInv(X, X').$$

Since $R(X) \rightarrow C(X)$, therefore $R(X) \wedge Tr(X, X') \rightarrow C(X) \wedge Tr(X, X')$, the base case holds.

Inductive step: Assume $R(X) \wedge Tr^n(X, \ldots, X') \rightarrow TrInv(X, X')$ for some $n \geq 1$. Let $Tr^{n+1}(X, X'')$ hold, so there $\exists X, \ldots, X', X''. R(X) \wedge Tr^n(X, \ldots, X') \wedge Tr(X', X'')$. Since $R(X) \wedge Tr^n(X, \ldots, X') \rightarrow TrInv(X, X')$, then trivially $R(X) \wedge Tr^n(X, X') \wedge Tr(X', X'') \rightarrow R(X) \wedge TrInv(X, X') \wedge Tr(X', X'')$. Now, due to the fact that $R(X) \rightarrow C(X)$, the following holds: $R(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow C(X) \wedge TrInv(X, X') \wedge Tr(X', X'')$. And by claim in the lemma:

$$C(X) \wedge TrInv(X, X') \wedge Tr(X', X'') \rightarrow TrInv(X, X'').$$

Therefore, through the chain of implications, we have:

$$R(X) \wedge Tr^{n+1}(X, X') \rightarrow TrInv(X, X').$$

Thus $TrInv$ is a disjunctively well-founded transition invariant over all reachable states, and TS is terminating. $\square$

B. Interpolation-based Termination Analysis

This subsection presents the full termination analysis procedure, using the interpolation-based transition invariant generation. It integrates TrInvGen into the safety-based analysis loop of Algorithm 1. At each iteration, instead of only blocking terminating states, the algorithm also constructs a transition invariant candidate. The overall flow is shown in Figure 3.

The Algorithm 4 takes as input a Safety Problem $P = \langle Init, Tr, Sink, X\rangle$, where $Init$ and Tr encode the initial states and transition relation, $Sink$ is the formula encoding sink states, and X is a set of variables over which the Transition System is defined. As an output, the algorithm returns NONTERM if TS is nonterminating, TERM if it is terminating, and UNKNOWN if the procedure cannot determine

Algorithm 4: Interpolation-based Transition Invariant Generation for Proving and Disproving Termination.

Input : Safety problem $P = \langle Init, Tr, Sink, X \rangle$
Output: TERM | NONTERM | UNKNOWN
Data : $TInv$ - formula that encodes transition invariant

1 $TInv \leftarrow \bot$
2 **while** $\top$ **do**
3 $\quad \langle res, \phi = Tr^n \ or \ Inv \rangle \leftarrow safetyCheck(P)$
4 $\quad$ **if** $res = UNSAFE$ **then**
5 $\quad\quad \langle TInv, C \rangle \leftarrow$ TRINVGEN$(P, Tr^n, TInv)$
6 $\quad\quad \langle res', \_ \rangle \leftarrow safetyCheck(\langle Init, Tr, \neg C, X \rangle)$
7 $\quad\quad$ **if** $res' = UNSAFE$ **then**
8 $\quad\quad\quad P \leftarrow$ BLOCKTERM(P, ϕ)
9 $\quad\quad$ **else**
10 $\quad\quad\quad$ **return** $TERM$
11 $\quad$ **else**
12 $\quad\quad$ **if** $\forall X. \phi(X) \rightarrow \exists X'. Tr(X, X') \wedge \phi(X')$ **then return** $NONTERM$
13 $\quad\quad$ **return** UNKNOWN

the final result. The procedure maintains a formula encoding a candidate transition invariant $TInv(X, X')$. During the execution, $TInv$ is extended with new disjuncts, which are generated by the TRINVGEN procedure, and encode well-founded relations. Changes to the original SNA are in the lines 5, 6, and 10.

Algorithm commences by analyzing the original safety problem. Every iteration, similarly to the Algorithm 1, begins with the safety check at line 2. The algorithm handles two possible cases:

a) UNSAFE.: A terminating trace Tr^n exists. The algorithm calls TRINVGEN to extend $TInv$ and compute the coverage formula $C(X)$ (line 5). Then, it checks if $\neg(C(X) \vee Sink(X))$ is reachable from the initial states (line 6). If $\neg(C(X) \vee Sink(X))$ is reachable, algorithm refines the safety problem by blocking the states deterministically leading to termination via BLOCKTERM, and the loop continues. Otherwise, $TInv$ is a transition invariant over all reachable states, and algorithm concludes termination.

b) SAFE.: No sink state is reachable. If $\phi(X)$ is a recurrent set, the system is nonterminating. Otherwise, the algorithm returns UNKNOWN as the analysis is inconclusive.

Example 4. Consider TS from Example 2 with $Init \triangleq x > 0$, $Tr \triangleq \big((x' = x-1 \vee x' = x-2) \wedge x > 0\big) \vee \big(x < -10 \wedge x' = x-1\big)$, $X \triangleq \{x\}$.

Preprocessing computes $Sink \triangleq x \leq 0 \wedge x \geq -10$. Then, the algorithm is called with $P = \langle Init, Tr, Sink, X \rangle$.

The safety check detects a trace Tr^1 reaching $Sink$. TRINVGEN produces a transition invariant candidate $TInv(X, X') \triangleq x' < x \wedge x' \geq 0$ and the formula $C(X) = x > 0$. The safety check for $\langle Init, Tr, \neg(C(X) \vee Sink(X)), x \rangle$ returns SAFE, and algorithm concludes that the transition system is terminating.

C. Correctness

This subsection proves the soundness of the introduced approach. We first prove that when algorithm returns TERM, the transition system is nonterminating, and then we prove the correctness of the nontermination.

Lemma 3. *If Algorithm 4 returns $TERM$, then the transition system $\langle Init, Tr, X \rangle$ is terminating.*

Proof. $TERM$ is returned only at the line 10, of the algorithm. It means that there exists a transition invariant $TInv$ which is valid over states $C(X)$, and $\neg C(X)$ is unreachable from the initial states. The safety problem is updated iteratively, blocking the states that deterministically lead to the termination, so in the end the $P = \langle Init', Tr', Sink, X \rangle$, where $Init'(X) = Init(X) \wedge \bigwedge_{i=0}^{m} \neg S_i(X)$ and $Tr' = Tr(X) \wedge \bigwedge_{i=0}^{l} \neg S_j(X')$. According to the Lemma 2, Transition System $\langle Init', Tr', X \rangle$ is terminating, since $TInv$ is a disjunctively well-founded transition invariant over all reachable states.

Since the states satisfying $\bigvee_{i=0}^{m} S_i(X)$ and $\bigvee_{i=0}^{l} S_j(X')$ deterministically lead to the termination, every trace of $\langle Init, Tr, X \rangle$ either follows Tr' or passes through a blocked state which leads to termination. Hence $\langle Init, Tr, X \rangle$ is terminating. $\square$

Lemma 4. *If Algorithm 4 returns $NONTERM$, then the transition system $\langle Init, Tr, X \rangle$ is nonterminating.*

Proof. The proof of this lemma follows directly from the Lemma 1, as the nontermination proof procedure directly corresponds to one described in the Algorithm 1. $\square$

Example 5. Consider a transition system $TS = \langle Init, Tr, X \rangle$ with $X = \{x, u\}$, where x, u are integer variables, $Init(X) \triangleq u < 10 \wedge x > 0$, and $Tr(X, X') \triangleq ((x' = x - 1 \wedge u' = u) \vee (x' = x + 1 \wedge u' = u + 1 \wedge u < 10)) \wedge x > 0$.

Intuitively, at each step the system either decrements x (and keeps u unchanged), or increments both x and u provided $u < 10$. Once $u = 10$, only the decrementing branch is available, and x decreases monotonically to zero. Hence TS is terminating. The sink states are $Sink(X) \triangleq x \leq 0$. The initial safety problem is $P = \langle Init, Tr, Sink, X \rangle$.

Iteration 1. $safetyCheck(P)$ returns UNSAFE with a trace $Tr^1(X^{(0)}, X^{(1)})$: a sink state is reachable in one transition via the decrement when $x = 1$. TRINVGEN constructs transition invariant: $TInv(X, X') \triangleq x' < x \wedge x > 0$. The coverage formula is then computed as: $C(X) \triangleq u = 10$. The safety check for $\langle Init, Tr, \neg C, X \rangle$ returns UNSAFE, since states with $u < 10$ are reachable from $Init$. The algorithm therefore calls BLOCKTERM, blocking the sink-reaching transition:

$$Tr(X, X') \leftarrow Tr(X, X') \wedge x' > 0.$$

Iteration 2. $safetyCheck(P)$ returns SAFE with inductive invariant $Inv(X) \triangleq x > 0$. Since $x = 1 \wedge u = 10$ has no outgoing transition under the refined Tr' (the decrementing branch would produce $x' = 0$, which is now blocked), Inv is not a recurrent set. Algorithm 4 returns UNKNOWN.

The limitation. The root cause why algorithm cannot prove termination is that $TInv$ does not cover states with $u < 10$. It captures why the system terminates when $u = 10$, but not the global argument: u grows until at most $u = 10$, after which

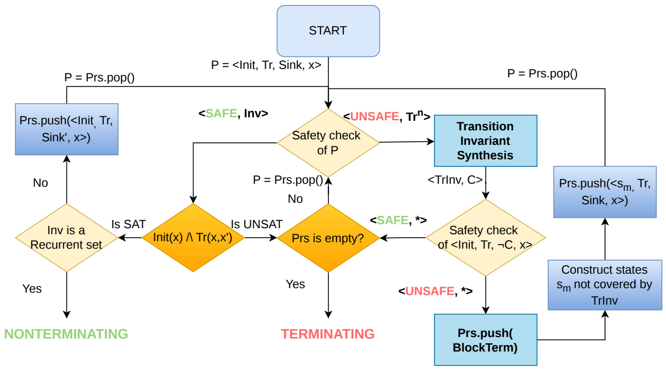

Figure 4. Optimized Interpolation-based Termination Analysis (New components are highlighted in dark orange and dark blue colours).

x decreases monotonically. A complete termination proof requires a *second* disjunct covering the $u < 10$ region. Below, we propose an extension which would allow to concentrate on the state space which is not covered by transition invariant.

IV. Extended Interpolation-based Transition Invariant Generation for Proving and Disproving Termination

Algorithm 4 is already capable of proving termination efficiently through the construction of transition invariants. This section describes an extension of the algorithm that aims to achieve better interaction between termination and nontermination analysis. Additionally, we present further extensions to the original SNA algorithm that improve overall performance. The high-level flow of the extended algorithm can be seen in Figure 4.

A. Utilising States for Which Transition Invariant is not Valid

Constructed transition invariant candidate can be used to guide the termination analysis. Particularly, the Algorithm 4 constructs the formula $C(X)$ encoding states for which the transition invariant is valid. If $\neg C(X)$ is reachable, it means that there exist states for which the transition invariant is not valid, and which potentially can lead to non-termination. The intuition is to compute the reachable states S_m within $\neg C(X)$ (which corresponds to line 16 of Algorithm 5). Then, to attempt either prove termination or nontermination of these particular states within considered transition system (line 17).

In the case of termination, the algorithm produces a witness transition invariant that extends the transition invariant of the original system, covering previously uncovered states. Otherwise, if nontermination is proven for S_m, then the original TS is also nonterminating, since S_m is reachable from the initial states.

B. Full Algorithm

Algorithm 5 extends Algorithm 4 to enable more efficient termination and nontermination proving. First, in addition to maintaining the transition invariant, the algorithm maintains a

Algorithm 5: Extended Interpolation-based Transition Invariant Generation for Proving and Disproving Termination.

Input : Safety problem $P = \langle Init, Tr, Sink, X\rangle$
Output: TERM | NONTERM
Data : $TInv$ - formula that encodes transition invariant, Prs - stack of Safety Problems to be analyzed for termination

1 $TInv \leftarrow \bot$
2 $Prs \leftarrow [P]$
3 **while** $\top$ **do**
4 $\quad \langle Init', Tr', Sink', X\rangle \leftarrow Prs.pop()$
5 $\quad \langle res, \phi = Inv\ or\ Tr^n\rangle \leftarrow safetyCheck(\langle Init', Tr', Sink', X\rangle)$
6 $\quad$ **if** $\langle res, \phi\rangle = \langle UNSAFE, Tr^n(X^{(0)}, \ldots, X^{(n)})\rangle$ **then**
7 $\quad\quad \langle TInv, C\rangle \leftarrow$ TrInvGen$(P, \phi, TInv)$
8 $\quad\quad \langle res', \phi'\rangle \leftarrow safetyCheck(\langle Init', Tr', \neg C, X\rangle)$
9 $\quad\quad$ **if** $res' = SAFE$ **then**
10 $\quad\quad\quad$ **if** $Prs = \emptyset$ **then**
11 $\quad\quad\quad\quad$ **return** $TERM$
12 $\quad\quad\quad$ **else**
13 $\quad\quad\quad\quad$ **continue**
14 $\quad\quad$ **else**
15 $\quad\quad\quad Prs.push($BlockTerm$(P', \phi))$
16 $\quad\quad\quad S_m \leftarrow QE(\exists X^{(0)}, \ldots, X^{(m-1)}, Init(X^{(0)}) \land \phi'(X^{(0)}, \ldots, X^{(m)}) \land \neg C(X^{(m)}))$
17 $\quad\quad\quad Prs.push(\langle S_m, Tr', Sink', X\rangle)$
18 $\quad$ **else**
19 $\quad\quad$ **if** $\forall X.\phi(X) \rightarrow \exists X'.Tr'(X, X') \land \phi(X')$ **then return** $NONTERM$
20 $\quad\quad$ **if** *UNSAT ?* $\phi(X) \land Tr'(X, X')$ **then**
21 $\quad\quad\quad$ **if** $Prs = \emptyset$ **then return** $TERM$
22 $\quad\quad$ **else**
23 $\quad\quad\quad Sink'(X) \leftarrow \neg QE(\exists X'.Tr'(X, X'))$
24 $\quad\quad\quad Prs.push(\langle Init', Tr', Sink', X\rangle)$

stack of safety problems to be analyzed: Prs (line 2). At each iteration, the algorithm pops a safety problem from the stack and analyzes it. The stack allows the algorithm to focus on the termination analysis of particular reachable states within the TS. The original safety problem always remains in the Prs stack with blocked states that deterministically lead to termination.

The postprocessing phase of transition invariant construction is the main change to the Algorithm 4. When $\neg C$ is reachable, the extended algorithm performs two key actions: (1) it pushes the BlockTerm-refined problem (as in the original algorithm), and (2) it additionally pushes a new subproblem where states not covered by the transition invariant (S_m, line 17) serve as the initial states. This approach allows the algorithm to concentrate on proving termination or nontermination for particular reachable states, rather than necessarily proving termination for the entire system.

Otherwise, if $\neg C$ is not reachable, $TInv$ proves termination of the currently analyzed TS (which may be a refined TS previously pushed onto the stack), and the number of remaining subproblems decreases.

The following subsections describe two additional optimizations that further enhance algorithm performance:

Simple termination check. When the safety check returns SAFE and $Inv(X)$ is not a recurrent set, the algorithm can additionally check whether it is possible to take a transition from the initial states by querying:

$$Init(X) \wedge Tr(X, X')$$

If this formula is unsatisfiable, then no transition can be taken from the initial states (or all transitions are blocked, meaning all reachable states deterministically lead to termination), and the system is trivially terminating. This check is implemented at line 21.

Refinement of Recurrent sets. When $Inv(X)$ is not a recurrent set, this is often because BLOCKTERM has eliminated all transitions leading to $Sink$. This means that produced Inv contains sink states itself, due to the updated Tr. New sink states can be computed as:

$$Sink'(X) = \neg QE(\exists X'.Tr'(X, X'))$$

The formula $Sink'$ replaces $Sink$ in the safety problem, enabling the analysis to continue rather than returning UNKNOWN. This extension is implemented at line 23.

Lemma 5. *If Algorithm 5 returns $TERM$, then the original transition system $\langle Init, Tr, X\rangle$ is also terminating.*

Proof. TERM is returned at line 11 or line 21. Similarly to Algorithm 4, to return $TERM$, the termination conditions need to be satisfied for the transition system $\langle Init', Tr', X\rangle$, where $Init'(X) = Init(X) \wedge \bigwedge_i \neg S_i(X)$, and $Tr'(X, X') = Tr(X, X') \wedge \bigwedge_j \neg S_j(X')$. $\bigvee_{i=0}^{m} S_i(X)$ and $\bigvee_{j=0}^{l} S_j(X')$ are the states that deterministically lead to termination.

Case 1. If $TERM$ is returned at line 11, then the termination proof follows directly from the Lemma 2.

Case 2. If $TERM$ is returned at line 21, then the formula $Init'(X) \wedge Tr'(X, X')$ is unsatisfiable, so no transition is reachable from $Init'$. This means that the transition system is terminating, since all feasible transitions in the original transition system lead to deterministically terminating states. □

Lemma 6. *If Algorithm 5 returns $NONTERM$, then the original transition system $\langle Init, Tr, X\rangle$ is nonterminating.*

Proof. NONTERM is returned at line 19, where $\phi(X)$ is a recurrent set for the current refined system $\langle Init', Tr', X\rangle$. By construction, $Init'$ is reachable from $Init$ (it is either $Init$ itself), and $Tr'(X, X') \rightarrow Tr(X, X')$. Based on Lemma 1, if $NONTERM$ is returned, then there exists a recurrent set in the transition system $\langle Init', Tr', X\rangle$, which means that there also exists a recurrent set in the original transition system $\langle Init, Tr, X\rangle$. This proves nontermination. □

V. Implementation and Evaluation

This section presents implementation details and describes the experimental evaluation of our termination approach.

A. Implementation

We implemented Algorithms 1, 4, and 5 inside of GOLEM [5]; we refer to these implementations as SNA, ITPTIG and ITPTIG+. GOLEM was chosen as it supports multiple different safety verification procedures, such as Property Directed Reachability [7] (PDR), Transition Power Abstraction [6] (TPA) and Interpolation-based Model Checking [25]. Additionally, we use quantifier elimination, already implemented in GOLEM.

For SMT queries and interpolant construction, our implementation uses the OPENSMT SMT solver [18]. OPENSMT is integrated within GOLEM and provides native support for interpolation.

B. Evaluation

We compare ITPTIG+ against SNA and ITPTIG, to evaluate the effect of combination of the termination and nontermination techniques. Additionally, we evaluate ITPTIG+ against several state-of-the-art tools: LOAT [24] (version eacb74b), KOAT [24] (version 41cb681), T2 [9] (version 10f1373). Evaluation focuses on termination analysis of Integer Transition Systems. Experiments were conducted on Ubuntu 20.04 machine with an AMD EPYC 7452 32-core processor and 8x32 GiB RAM. Our evaluation addresses the following research questions:

- **RQ1:** How does extending SNA with the termination procedure affect verification performance?
- **RQ2:** How does ITPTIG+ perform compared to state-of-the-art tools?

The benchmark suite of Integer Transition Systems is taken from the Termination Competition. Overall there are 1180 benchmarks (538 nonterminating, 623 terminating, and 19 unknown). The evaluation was executed with 120 seconds timeout (increase of the timeout does not significantly influence the evaluation).

Algorithm	Total	Terminating	Nonterminating	Uniquely
SNA	343	0	343	7
ITPTIG	521	186	335	7
ITPTIG+	761	320	441	230

Table I
COMPARISON OF ITPTIG+ WITH SNA AND ITPTIG.

Table I gives the experimental results for ITPTIG+, SNA, and ITPTIG. Our results indicate that ITPTIG+ consistently outperforms both SNA and ITPTIG for Terminating and Nonterminating instances. We attribute this to the generation "subproblems" in the ITPTIG+, that allows the algorithm to focus on the termination of specific subproblem instead of considering the whole system. This benefits recurrent set detection, by focusing the analysis on specific states not covered by the transition invariant. It also benefits termination proving, by enriching the transition invariant with additional disjuncts that cover different regions of the reachable state space. In answer to **RQ1**, the combination of termination and nontermination reasoning in ITPTIG+ yields significantly

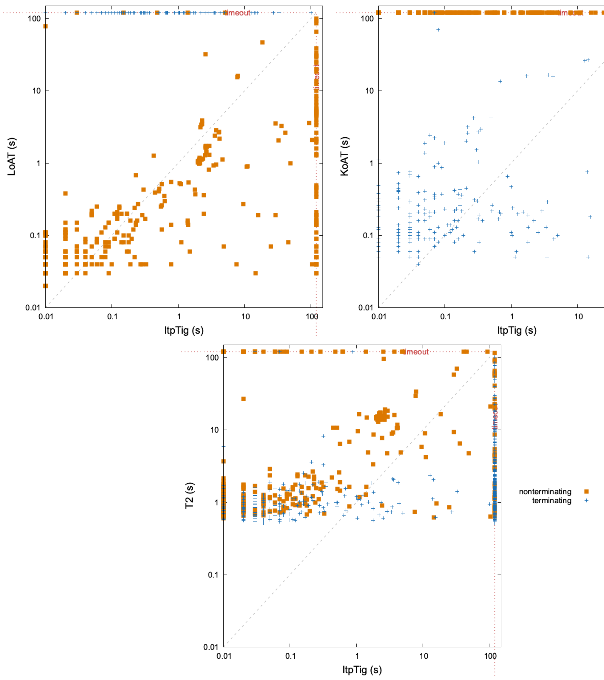

Figure 5. Comparison of ITPTIG+ with state-of-the-art tools.

improved performance across both problem classes. Notably, during experimentation we noticed that ITPTIG+ produces different outputs depending on the underlying Safety Verification engine. For example, the most efficient results were produced using TPA ITPTIG+- 755 instances solved. ITPTIG+ with PDR solves less instances (751), however it can solve 17 benchmarks uniquely compared to the TPA.

Figure 5 shows a scatter plot comparing ITPTIG+ against state-of-the-art termination analysers. Orange squares denote nonterminating instances; blue crosses denote terminating instances. Points above the diagonal indicate instances where ITPTIG+ is faster than the competitor. ITPTIG+ is able to solve 755 benchmarks, with T2 solving 1002, KOAT 562, and LOAT 525 instances. As shown in Figure 5, ITPTIG+ is competitive with state-of-the-art termination analysers. Among all tools, ITPTIG+ uniquely solves 8 instances. Notably, 2 of these — `juLinkedListCreateAddAll.jar-obl-11` and `juLinkedListCreateAddAllAt.jar-obl-17` — belong to the 19 previously unsolved instances in the Termination Competition, for which ITPTIG+ finds recurrent sets proving nontermination. In answer to **RQ2**, ITPTIG+ is competitive with state-of-the-art tools overall and uniquely solves instances previously unsolved in the Termination Competition, demonstrating the practical value of the interpolation-based transition invariant approach.

VI. Related Work

Termination and nontermination analysis are very active areas of research. We will discuss techniques, related to Interpolation-based Termination Analysis below.

Algorithm	Total	Terminating	Nonterminating	Uniquely
LOAT	525	0	525	48
KOAT	562	562	0	26
T2	1002	572	430	40
ITPTIG+	761	320	441	8

Table II
COMPARISON OF ITPTIG+ WITH STATE-OF-THE-ART TOOLS

Nontermination. Nontermination analysis is typically centered around the construction of recurrent sets — sets of states from which execution cannot escape [1], [15], [9], [11], [10], [21]. These techniques can vary from the template-based synthesis to the safety-check based recurrent set construction. Another traditional approach is safety-based nontermination analysis [4], that detects nontermination if the same state is reachable within TS more then ones - detecting a lasso. Newly introduced nontermination technique is the construction of geometric nontermination argument [23] - a finite representation of an infinite execution that via a sum of geometric series.

Termination. There exist many different techniques for the termination analysis of software. One of the most researched and applied techniques is the synthesis of the ranking function [26], [20], [2], [17], [16], [28], [1], [31]. State of the art techniques are centered on the construction of multiphase or lexicographic ranking functions. These complex techniques significantly broaden the class of programs for which the termination can be proven. However, the complexity of transition system can make construction of ranking function difficult. Other technique, which allows to efficiently prove termination are fixpoint computation [30], which relies on the reduction of termination/nontermination problems into first order fixpoint formula and checking its validity.

There also exists another set of techniques that are based on the construction of well-founded transition invariants [**?**], [12], [19], [29]. These approaches overapproximate the behaviour of the transition system, simplifying the termination proof. The core problem of these approaches is the generation of transition invariant. Listed techniques rely either on the template-based generation or on the direct generation of ranking relation based on a particular trace in the transition system. In contrast, we propose to do the guided generation of transition invariants, based on the terminating traces and the actual termination conditions. Additionally, our technique is combined with a nontermination approach, increasing the efficiency of both techniques.

VII. Conclusion

This paper introduces Termination Analysis via Interpolation-based Synthesis of Transition Invariants. A novel approach for the termination analysis that extends Safety-based Nontermination Analysis technique. Our key insight is to use interpolation for the construction of termination witnesses, that allows the transition invariant to be guided by termination condition. Additionally, our technique is strongly centered on the interaction between termination and nontermination approaches, using transition invariants to limit the explored state space and using safety-based nontermination to generate candidate transition invariants. Application of safety verifiers provides a strong background for the analysis, allowing to quickly check termination and nontermination. We proved the soundness of the algorihtm and conducted an extensive experimental evaluation, which demonstrates both the eficiency of the combination of termination and nontermination techniques, and the competitive performance compared to other state-of-the-art termination analysis tools.

This work was funded by SNCF grant .... We also want to express gratitude to Marek Jankola and Dirk Beyer for the fruitful conversations on the topic of termination.

References

[1] Ben-Amram, A.M., Doménech, J.J., Genaim, S.: Multiphase-linear ranking functions and their relation to recurrent sets. In: Chang, B.E. (ed.) Static Analysis - 26th International Symposium, SAS 2019, Porto, Portugal, October 8-11, 2019, Proceedings. pp. 459–480. Lecture Notes in Computer Science, Springer (2019). https://doi.org/10.1007/978-3-030-32304-2_22, https://doi.org/10.1007/978-3-030-32304-2_22

[2] Ben-Amram, A.M., Genaim, S.: Ranking functions for linear-constraint loops. J. ACM **61**(4), 26:1–26:55 (2014). https://doi.org/10.1145/2629488, https://doi.org/10.1145/2629488

[3] Beyer, D., Jankola, M., Lingsch-Rosenfeld, M.: Transition invariants revisited: Termination witnesses and their validation. In: Proc. CAV. Springer (2026), https://www.sosy-lab.org/research/pub/2026-CAV.Transition_Invariants_Revisited_Termination_Witnesses_and_Their_Validation.pdf

[4] Biere, A., Artho, C., Schuppan, V.: Liveness checking as safety checking. In: Cleaveland, R., Garavel, H. (eds.) 7th International ERCIM Workshop in Formal Methods for Industrial Critical Systems, FMICS 2002, ICALP 2002 Satellite Workshop, Málaga, Spain, July 12-13, 2002. pp. 160–177. No. 2 in Electronic Notes in Theoretical Computer Science, Elsevier (2002). https://doi.org/10.1016/S1571-0661(04)80410-9, https://doi.org/10.1016/S1571-0661(04)80410-9

[5] Blicha, M., Britikov, K., Sharygina, N.: The golem horn solver. In: Enea, C., Lal, A. (eds.) Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17-22, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 13965, pp. 209–223. Springer (2023), https://doi.org/10.1007/978-3-031-37703-7_10

[6] Blicha, M., Fedyukovich, G., Hyvärinen, A.E.J., Sharygina, N.: Transition power abstractions for deep counterexample detection. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022. Lecture Notes in Computer Science, vol. 13243, pp. 524–542. Springer (2022), https://doi.org/10.1007/978-3-030-99524-9_29

[7] Bradley, A.R., Manna, Z.: Property-directed incremental invariant generation. Formal Aspects Comput. **20**(4-5), 379–405 (2008). https://doi.org/10.1007/S00165-008-0080-9, https://doi.org/10.1007/s00165-008-0080-9

[8] Braverman, M.: Termination of integer linear programs. In: Ball, T., Jones, R.B. (eds.) Computer Aided Verification, 18th International Conference, CAV 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings. pp. 372–385. Lecture Notes in Computer Science, Springer (2006). https://doi.org/10.1007/11817963_34, https://doi.org/10.1007/11817963_34

[9] Brockschmidt, M., Cook, B., Ishtiaq, S., Khlaaf, H., Piterman, N.: T2: temporal property verification. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 387–393. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_22, https://doi.org/10.1007/978-3-662-49674-9_22

[10] Brockschmidt, M., Ströder, T., Otto, C., Giesl, J.: Automated detection of non-termination and nullpointerexceptions for java bytecode. In: Beckert, B., Damiani, F., Gurov, D. (eds.) Formal Verification of Object-Oriented Software - International Conference, FoVeOOS 2011, Turin, Italy, October 5-7, 2011, Revised Selected Papers. pp. 123–141. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-31762-0_9, https://doi.org/10.1007/978-3-642-31762-0_9

[11] Chen, H.Y., Cook, B., Fuhs, C., Nimkar, K., O'Hearn, P.W.: Proving nontermination via safety. In: Ábrahám, E., Havelund, K. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings. pp. 156–171. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_11, https://doi.org/10.1007/978-3-642-54862-8_11

[12] Cook, B., Podelski, A., Rybalchenko, A.: Abstraction refinement for termination. In: Hankin, C., Siveroni, I. (eds.) Static Analysis, 12th International Symposium, SAS 2005, London, UK, September 7-9, 2005, Proceedings. pp. 87–101. Lecture Notes in Computer Science, Springer (2005). https://doi.org/10.1007/11547662_8, https://doi.org/10.1007/11547662_8

[13] Cook, B., Podelski, A., Rybalchenko, A.: Termination proofs for systems code. In: Schwartzbach, M.I., Ball, T. (eds.) Proceedings of the ACM SIGPLAN 2006 Conference on Programming Language Design and Implementation, Ottawa, Ontario, Canada, June 11-14, 2006. pp. 415–426. ACM (2006). https://doi.org/10.1145/1133981.1134029, https://doi.org/10.1145/1133981.1134029

[14] Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. The Journal of Symbolic Logic **22**(3), 269–285 (1957)

[15] Frohn, F., Giesl, J.: Proving non-termination and lower runtime bounds with loat (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) Automated Reasoning - 11th International Joint Conference, IJCAR 2022, Haifa, Israel, August 8-10, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13385, pp. 712–722. Springer (2022). https://doi.org/10.1007/978-3-031-10769-6_41, https://doi.org/10.1007/978-3-031-10769-6_41

[16] Giesl, J., Aschermann, C., Brockschmidt, M., Emmes, F., Frohn, F., Fuhs, C., Hensel, J., Otto, C., Plücker, M., Schneider-Kamp, P., Ströder, T., Swiderski, S., Thiemann, R.: Analyzing program termination and complexity automatically with aprove. J. Autom. Reason. **58**(1), 3–31 (2017). https://doi.org/10.1007/S10817-016-9388-Y, https://doi.org/10.1007/s10817-016-9388-y

[17] Heizmann, M., Hoenicke, J., Podelski, A.: Termination analysis by learning terminating programs. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 797–813. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_53, https://doi.org/10.1007/978-3-319-08867-9_53

[18] Hyvärinen, A.E.J., Marescotti, M., Alt, L., Sharygina, N.: Opensmt2: An SMT solver for multi-core and cloud computing. In: Creignou, N., Berre, D.L. (eds.) Theory and Applications of Satisfiability Testing - SAT 2016 - 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9710, pp. 547–553. Springer (2016). https://doi.org/10.1007/978-3-319-40970-2_35, https://doi.org/10.1007/978-3-319-40970-2_35

[19] Kroening, D., Sharygina, N., Tsitovich, A., Wintersteiger, C.M.: Termination analysis with compositional transition invariants. In: Touili, T., Cook, B., Jackson, P.B. (eds.) Computer Aided Verification, 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings. pp. 89–103. Lecture Notes in Computer Science, Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_9, https://doi.org/10.1007/978-3-642-14295-6_9

[20] Kura, S., Unno, H., Hasuo, I.: Decision tree learning in cegis-based termination analysis. In: Silva, A., Leino, K.R.M. (eds.) Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II. pp. 75–98. Lecture Notes in Computer Science, Springer (2021). https://doi.org/10.1007/978-3-030-81688-9_4, https://doi.org/10.1007/978-3-030-81688-9_4

[21] Larraz, D., Nimkar, K., Oliveras, A., Rodríguez-Carbonell, E., Rubio, A.: Proving non-termination using max-smt. In: Biere, A., Bloem, R. (eds.) Computer Aided Verification - 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18-22, 2014. Proceedings. pp. 779–796. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_52, https://doi.org/10.1007/978-3-319-08867-9_52

[22] Le, T.C., Qin, S., Chin, W.: Termination and non-termination specification inference. In: Grove, D., Blackburn, S.M. (eds.) Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015. pp. 489–498. ACM (2015). https://doi.org/10.1145/2737924.2737993, https://doi.org/10.1145/2737924.2737993

[23] Leike, J., Heizmann, M.: Geometric nontermination arguments. In: Beyer, D., Huisman, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 24th International Conference, TACAS 2018, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2018, Thessaloniki, Greece, April 14-20, 2018, Proceedings, Part II. pp. 266–283. Lecture Notes in Computer Science, Springer (2018). https://doi.org/10.1007/978-3-319-89963-3_16, https://doi.org/10.1007/978-3-319-89963-3_16

[24] Lommen, N., Giesl, J.: Aprove (koat + loat). In: Gurfinkel, A., Heule, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 205–211. Springer Nature Switzerland, Cham (2025)

[25] McMillan, K.L.: Interpolation and sat-based model checking. In: Jr., W.A.H., Somenzi, F. (eds.) Computer Aided Verification, 15th International Conference, CAV 2003, Boulder, CO, USA, July 8-12, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2725, pp. 1–13. Springer (2003), https://doi.org/10.1007/978-3-540-45069-6_1

[26] Podelski, A., Rybalchenko, A.: A complete method for the synthesis of linear ranking functions. In: Steffen, B., Levi, G. (eds.) Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings. pp. 239–251. Lecture Notes in Computer Science, Springer (2004). https://doi.org/10.1007/978-3-540-24622-0_20, https://doi.org/10.1007/978-3-540-24622-0_20

[27] Podelski, A., Rybalchenko, A.: Transition invariants. In: 19th IEEE Symposium on Logic in Computer Science (LICS 2004), 14-17 July 2004, Turku, Finland, Proceedings. pp. 32–41. IEEE Computer Society (2004). https://doi.org/10.1109/LICS.2004.1319598, https://doi.org/10.1109/LICS.2004.1319598

[28] Sarita, Y., Singh, A., Gomber, S., Singh, G., Vishwanathan, M.: Efficient ranking function-based termination analysis via bidirectional decompositional search. In: Krebbers, R. (ed.) Programming Languages and Systems - 35th European Symposium on Programming, ESOP 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11-16, 2026, Proceedings, Part II. pp. 181–209. Lecture Notes in Computer Science, Springer (2026). https://doi.org/10.1007/978-3-032-22723-2_7, https://doi.org/10.1007/978-3-032-22723-2_7

[29] Tsitovich, A., Sharygina, N., Wintersteiger, C.M., Kroening, D.: Loop summarization and termination analysis. In: Abdulla, P.A., Leino, K.R.M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 17th International Conference, TACAS 2011, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2011, Saarbrücken, Germany, March 26-April 3, 2011. Proceedings. pp. 81–95. Lecture Notes in Computer Science, Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_9, https://doi.org/10.1007/978-3-642-19835-9_9

[30] Unno, H., Terauchi, T., Gu, Y., Koskinen, E.: Modular primal-dual fixpoint logic solving for temporal verification. Proc. ACM Program. Lang. **7**(POPL), 2111–2140 (2023). https://doi.org/10.1145/3571265, https://doi.org/10.1145/3571265

[31] Urban, C., Gurfinkel, A., Kahsai, T.: Synthesizing ranking functions from bits and pieces. In: Chechik, M., Raskin, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 22nd International Conference, TACAS 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2-8, 2016, Proceedings. pp. 54–70. Lecture Notes in Computer Science, Springer (2016). https://doi.org/10.1007/978-3-662-49674-9_4, https://doi.org/10.1007/978-3-662-49674-9_4